The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Khengar Chaun	24BCE10801
Slot	**Date of Submission**
A11	31/03/2026

Khengar Chaun (24BCE10801)

1. Origin and Philosophy:

Back in the day during the early 1990s. The web was just waking up, and almost everyone was using this one piece of software called the NCSA HTTPd daemon to keep their servers alive. It was the absolute backbone of the early internet. But then, plot twist happened that the main developer just packed up and left the project. Suddenly, the most important piece of web infrastructure was completely abandoned.

Webmasters everywhere were left frustrated. To keep their sites from crashing, they started writing their own DIY patches and essentially duct-taping their servers together. Because everyone was fixing things in isolation, the web was literally splintering into a bunch of incompatible pieces. So, in 1995, eight of these stressed-out webmasters got together over email. Instead of fighting it out, they decided to pool all their custom fixes into one giant, unified patch to save the internet. Because the software was basically just a bunch of patches stitched together, they joked that it was "a patchy" server—which became Apache. They didn't lock it behind a corporate paywall; they shared it openly because they knew the internet would collapse if there wasn't a solid, free foundation everyone could build on.

So, how do you legally manage a massive community project like that? You use the Apache License 2.0. It's what the industry calls a "permissive" license, which basically means, "Do whatever you want with this, just give us credit and don't sue us." It hands you the four golden freedoms of open source: you can run the software for absolutely any purpose, study its guts to see how it works, share copies with anyone, and distribute your own customized versions.

Here's where it gets interesting, especially when you compare it to a stricter license like the GPL. GPL is a "copyleft" license, acting kind of like a strict parent—if you modify GPL code and release it, you *must* share your new source code with the world to keep the community open. The Apache License is way more laid back. A massive tech giant can legally take the free Apache code, tweak it to make it incredibly fast, slap a price tag on it, and keep their new code a complete corporate secret without ever giving back to the community.

If I were putting my own code out there, my choice of license would totally depend on my end goal. If I built a useful tool and just wanted massive adoption across the corporate world with zero friction, I'd throw an Apache or MIT license on it. But if I built something that I wanted to guarantee would always stay in the hands of the community, I'd lock it down with GPL so nobody could hoard the improvements.

Khengar Chaun (24BCE10891)

At the end of the day, open-source philosophy comes down to understanding the difference between "free as in free beer" and "free as in freedom." Think about someone handing you the keys to a free car. That is "free beer"—you didn't spend a single rupee on it, but the hood is welded shut. You can drive it, but you can't open it up to see how the engine works or modify it. The Apache HTTP Server is "free freedom." It doesn’t just cost zero money; it gives you the ultimate liberty to pop the hood, rip apart the engine, see exactly how it runs, and completely rebuild it if you want to.

Open source isn't just about picking a software license; it’s basically a whole philosophy about how knowledge and tools ought to be shared. But should every single piece of software in the world be open source? Honestly, there's a strong argument for yes. When code is open, bugs get squashed instantly because thousands of developers are looking at it, and it stops mega-corporations from locking us into debt stack. But on the flip side, we have to be realistic. Developers need to pay their rent. Building ridiculously complex stuff like hospital management networks or high-end rendering software takes serious funding. If everything is completely free, getting investors to back massive, risky projects gets a whole lot harder.

Then there's the big elephant in the room: is it actually ethical for huge companies like Google or Meta to take free community labor, build a product on top of it, and make billions?. It definitely feels a bit exploitative when a massive tech giant cashes in on code maintained by some burnt-out volunteer in their free time without giving a dime back. But legally and ethically speaking, permissive licenses were designed to give them the green light to do exactly that. Plus, to be fair, a lot of these big players do end up hiring those same open-source developers or paying for the massive server costs to keep the projects afloat, so it isn't always just a one-way street.

In the software world, people throw around the phrase "standing on the shoulders of giants" all the time. What that really means is that nobody sits down today and codes a web server or an operating system completely from scratch. When someone builds a wild new app, they are probably dropping it on a Linux box, running it through Apache, and writing it in Python. They are literally standing on top of decades of unpaid, genius-level labor. Because of this, open source absolutely supercharges original innovation rather than undermining it. Since developers don't have to waste five years reinventing the wheel—like figuring out how basic internet

Khongar Chain (24BCE10001)

traffic routes—they can just start at step ten and focus all their brainpower on building something completely new on top of that solid foundation.

Khengar Chaun (24BCE10801)

2. FOSS Ecosystem:

No software exists in a vacuum, and Apache is essentially the grandfather of the modern web ecosystem. Because it is such a foundational piece of software, it tries to keep its dependencies as minimal as possible. It is written in C and compiled with standard open-source compilers like GCC. Its biggest dependencies are the Apache Portable Runtime (APR) library, which helps it run across different operating systems, PCRE for handling complex URL rewrites, and OpenSSL to manage secure HTTPS connections.

When you look at what Apache has enabled, the list is basically the entire early internet. The initial dot-com boom, early WordPress blogs, and giants like Wikipedia all scaled up using Apache. It also directly inspired its own biggest rival: Nginx. Nginx was created specifically to solve the "C10K problem" (handling 10,000 concurrent connections) because older, process-heavy versions of Apache struggled under that much heavy traffic.

In terms of web architecture, Apache is literally the "A" in the legendary LAMP stack (Linux, Apache, MySQL, PHP). It sits right in the middle of the workflow. Linux provides the operating system, Apache acts as the bouncer listening to the internet for incoming HTTP requests, it hands the request off to PHP and MySQL to build the dynamic webpage, and then Apache serves that finished HTML back to the user's browser.

As for who actually runs it, the software is governed by the Apache Software Foundation (ASF), which is a massive non-profit organization. They do not have a traditional corporate boss; they run on a strict meritocracy known as "The Apache Way." You only earn the right to vote on project decisions by contributing good code and earning the community's respect. The developers coordinate everything through massive, public mailing lists, bug trackers, and global developer conferences like ApacheCon.

Khengar Chaun (24BCE10801)

3. Open Source vs Proprietary (Analysis):

Dimension	Open Source	Proprietary Alternative
Cost	Absolutely $0. You can deploy it on ten thousand servers and it will never cost you a dime in licensing.	Expensive. IIS itself is "included" with Windows Server, but you have to buy the Windows Server OS license, which can cost hundreds or thousands of dollars depending on your server size and user count.
Security (who can audit the code?)	Anyone. The source code is public, meaning thousands of independent security researchers, hackers, and university students are constantly auditing the code for vulnerabilities.	Only Microsoft. The code is a black box. You just have to blindly trust that Microsoft's internal security team caught all the bugs before releasing it.
Support and reliability	Community-driven. You rely on documentation, Stack Overflow, and forums. If the server crashes at 3 AM, you are Googling the error code yourself (unless you hire a third-party consultant).	Corporate-backed. If you pay for the enterprise license, you get a direct phone line to Microsoft support. If it breaks, it is literally their job to help you fix it.
Freedom to modify	Total freedom. You can rip open the core C code, change how the server handles HTTP requests, and recompile it to fit your exact custom needs.	Zero freedom. The software is compiled and locked. You get what Microsoft gives you, and trying to reverse-engineer it violates the terms of service.
Community vs corporate control	Completely community-governed by the Apache Software Foundation. Decisions are made democratically by developers who earned their voting rights through hard work. No single CEO can wake up one day and decide to kill the project or lock a feature behind a paywall.	100% controlled by Microsoft executives and product managers. The roadmap is dictated by corporate profit goals. If Microsoft decides to force a massive upgrade or retire a feature you rely on, you have absolutely zero say in the matter.
Your overall verdict	The ultimate, battle-tested workhorse. It is the absolute best choice for flexibility, keeping budgets at zero, and running reliable web infrastructure without corporate interference.	A solid choice only if a company is already deeply locked into the Windows Server ecosystem and has the massive budget required to pay for official enterprise support. Too restrictive for general use.

Looking at all the facts, I would absolutely recommend the Apache HTTP Server for a real-world deployment. When you are building out web infrastructure, reliability and cost are everything. Apache has literally been battle-tested by the entire internet for decades, so you know it can handle heavy traffic. Plus, deploying it on a free Linux distribution instead of paying for massive Windows Server licenses frees up a huge amount of the budget that a company could instead spend on hiring better

developers or improving their actual product. Microsoft IIS is great if you are locked into a massive corporate ecosystem that demands official phone support, but for the vast majority of web projects, Apache is the smarter, leaner choice.

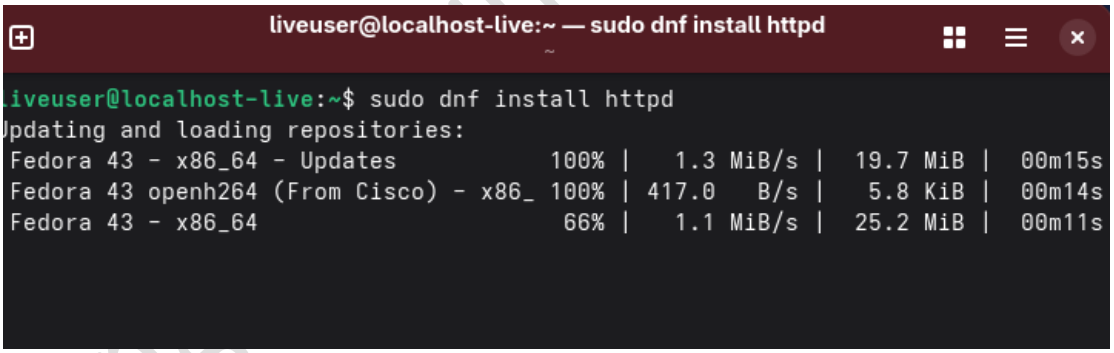
As for contributing to the project, I would definitely be willing to step up. I might not be a master at writing low-level C code to optimize its memory management right now, but the open-source ecosystem needs more than just hardcore programmers. I could contribute by writing better documentation, testing out new release patches on different Linux environments, or even just hanging out in the community forums answering setup questions for beginners. That is the beauty of the "Apache Way"—every little bit of community labor helps keep the web running.

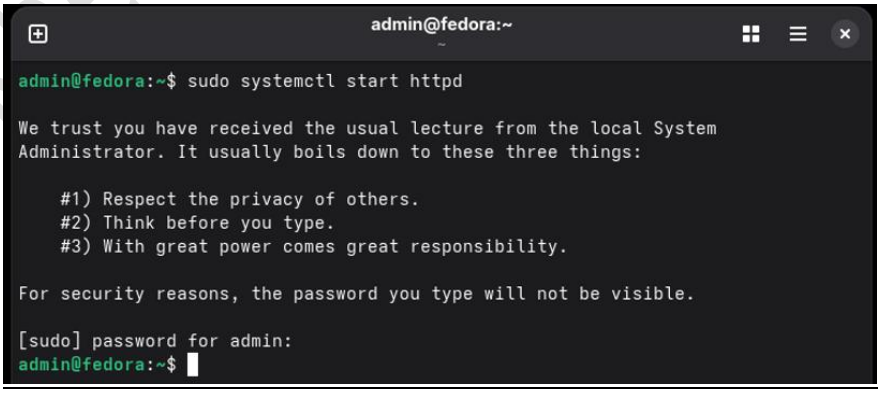
Khengar Chaun (24BCE10801)

4. Linux footprints:

To deploy the Apache HTTP Server, I used a Fedora Linux environment. Because Fedora is a RedHat-based distribution, the Apache package is officially named httpd rather than apache2. I initiated the installation using the command sudo dnf install httpd, utilizing Fedora's default package manager to pull the software from the official open-source repositories. Once installed, I managed the server's lifecycle—starting, enabling, and checking its status—using the systemctl command to interact with the systemd service manager.

During my footprint analysis, I verified that the core configuration files for the server are housed in /etc/httpd, while all access and error logs are securely routed to /var/log/httpd. From a security standpoint, it is critical to note that while the master process is initiated by the root user to bind to port 80, the actual worker threads that process internet traffic are run under a restricted, unprivileged user named apache. This isolation ensures that if a malicious actor ever manages to exploit a vulnerability in the web server, they are trapped in that restricted account and cannot gain root access to destroy the broader operating system.





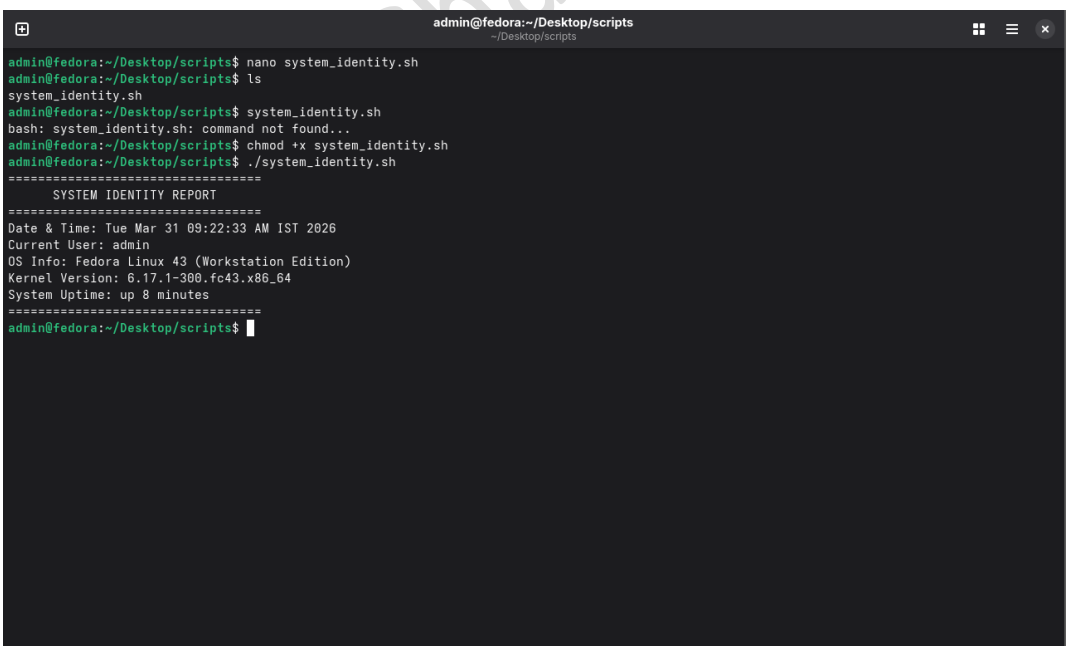
5. Shell scripts:

a. System Identity Report:

This script acts as a quick diagnostic dashboard. I built it to pull the currently logged-in account using the $USER environment variable. To grab the exact operating system details, the script uses grep to parse the /etc/os-release file and extract the Fedora version. It then communicates directly with the system to print the kernel version (uname -r) and calculates how long the server has been online (uptime -p).

```
#!/bin/bash

echo "==================================="
echo "      SYSTEM IDENTITY REPORT      "
echo "==================================="
echo "Date & Time: $(date)"
echo "Current User: $USER"
echo "OS Info: $(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d
'\"')"
echo "Kernel Version: $(uname -r)"
echo "System Uptime: $(uptime -p)"
echo "==================================="
```



b. **<u>FOSS Package Inspector:</u>**

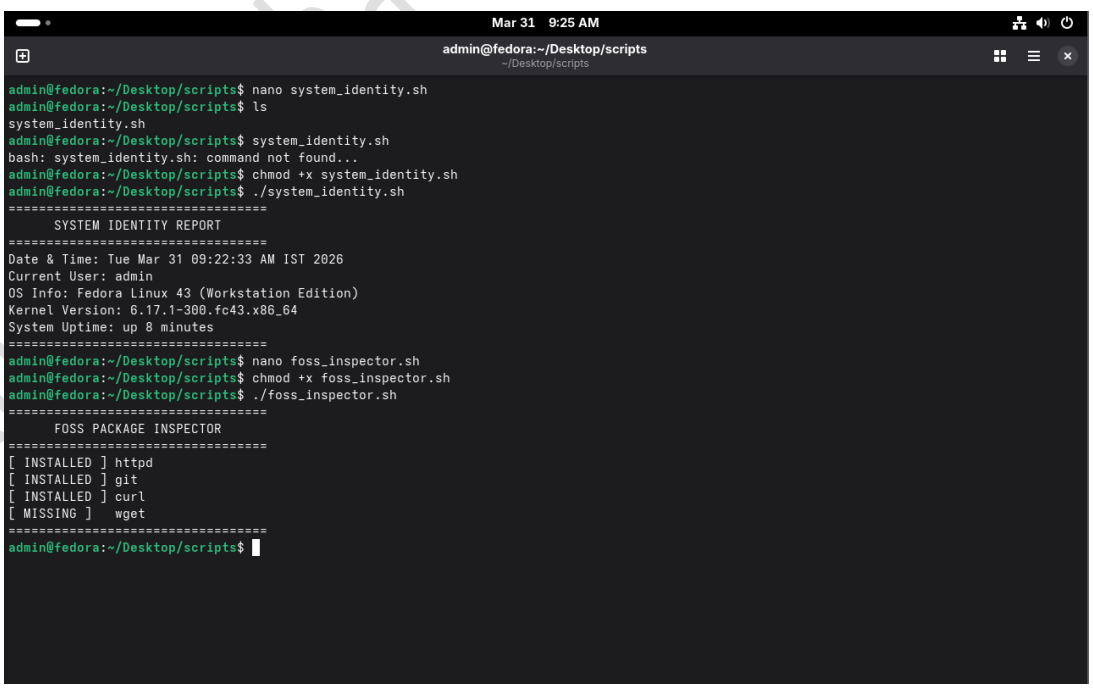
I designed this script to automate the auditing of essential server software. It stores a list of critical open-source tools (including httpd, git, and curl) inside a Bash array. A for loop feeds each package name into the rpm -q command to query the RedHat Package Manager. The script redirects the raw, messy terminal output to /dev/null, relying entirely on the silent exit code to cleanly print whether the software is "[INSTALLED]" or "[MISSING]".

```
#!/bin/bash

echo "==================================="
echo "      FOSS PACKAGE INSPECTOR       "
echo "==================================="

PACKAGES=("httpd" "git" "curl" "wget")

for pkg in "${PACKAGES[@]}"; do
    if rpm -q "$pkg" &> /dev/null; then
        echo "[ INSTALLED ] $pkg"
    else
        echo "[ MISSING ]   $pkg"
    fi
done
echo "==================================="
```



c. **<u>Disk Space Auditor:</u>**

This script functions as a safety monitor to ensure the server doesn't crash from a full hard drive. It runs df -h to list the disk filesystems in a human-readable format. The core logic extracts the exact usage percentage of the root drive (/) using awk, and strips away the % symbol using sed so it can be read as a pure integer. An if statement then evaluates that number, printing a healthy status if it is low, or triggering a critical warning if the drive exceeds 80% capacity.

```
#!/bin/bash

echo "==================================="
echo "        DISK SPACE AUDITOR         "
echo "==================================="
echo "Checking top 5 filesystems..."
echo ""
df -h | head -n 6
echo ""
echo "==================================="

USAGE=$(df -h / | awk 'NR==2 {print $5}' | sed 's/%//')

if [ "$USAGE" -gt 80 ]; then
    echo "[ WARNING ] Root partition is over 80% full! ($USAGE%)"
else
    echo "[ OK ] Root partition usage is healthy at $USAGE%."
fi
echo "==================================="
```

```
admin@fedora:~/Desktop/scripts$ nano disk_auditor.sh
admin@fedora:~/Desktop/scripts$ chmod +x disk_auditor.sh
admin@fedora:~/Desktop/scripts$ ./disk_auditor.sh
==================================
        DISK SPACE AUDITOR
==================================
Checking top 5 filesystems...

Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p3   38G  3.8G   34G  10% /
devtmpfs        3.9G     0  3.9G   0% /dev
tmpfs           3.9G   92K  3.9G   1% /dev/shm
tmpfs           1.6G  4.7M  1.6G   1% /run
tmpfs           1.0M     0  1.0M   0% /run/credentials/systemd-journald.service

==================================
[ OK ] Root partition usage is healthy at 10%.
==================================
admin@fedora:~/Desktop/scripts$
```

d. **Log analyser:**

I wrote this troubleshooting script to automatically filter out system noise when debugging a crashed server. It uses journalctl -n 200 to grab the last 200 system logs, and then pipes that text directly into grep. By applying the -iE flags, the script performs a case-insensitive search specifically for the keywords "error" or "failed". Finally, it passes those results through tail -n 5 to isolate and display only the five most recent critical warnings.

```
#!/bin/bash

echo "==================================="
echo "          LOG ANALYZER             "
echo "==================================="
echo "Scanning the last 200 system log entries for errors..."
echo ""

journalctl -n 200 --no-pager | grep -iE "error|failed" | tail -n 5

echo ""
echo "==================================="
echo "[ DONE ] Log analysis complete."
```

```
admin@fedora:~/Desktop/scripts$ ./log_analyzer.sh
==================================
          LOG ANALYZER
==================================
Scanning the last 200 system log entries for errors...

Mar 31 09:30:11 fedora gnome-text-editor[6265]: libEGL warning: failed to get driver name for fd -1
Mar 31 09:30:11 fedora gnome-text-editor[6265]: libEGL warning: MESA-LOADER: failed to retrieve device information
Mar 31 09:30:11 fedora gnome-text-editor[6265]: libEGL warning: failed to get driver name for fd -1
Mar 31 09:30:11 fedora gnome-text-editor[6265]: MESA: error: ZINK: failed to choose pdev
Mar 31 09:30:11 fedora gnome-text-editor[6265]: libEGL warning: egl: failed to create dri2 screen

==================================
[ DONE ] Log analysis complete.
admin@fedora:~/Desktop/scripts$
```

e. **<u>Manifesto generator:</u>**

This script demonstrates automated file creation and input/output redirection. Instead of opening a text editor manually, the script uses a "Here-Document" block (cat << 'EOF') to define a multi-line string containing the core principles of open-source software. The > operator redirects that entire block of text straight into a newly generated file called foss_manifesto.txt. To prove the operation was successful, the script instantly calls cat on the new file to print its contents back to the terminal.

```
#!/bin/bash

FILE="foss_manifesto.txt"

echo "==================================="
echo "       MANIFESTO GENERATOR        "
echo "==================================="
echo "Generating Open Source Manifesto..."

cat << 'EOF' > $FILE
THE OPEN SOURCE MANIFESTO
-------------------------
1. Free as in Freedom: Users have the liberty to run, copy, distribute, study, change, and improve the software.
2. Community Driven: The power of the software comes from the collective effort of developers worldwide.
3. Transparency: Code is open to audit, ensuring better security and no hidden backdoors.
4. No Corporate Monopolies: Software should not be locked behind proprietary walls that stifle innovation.
EOF

echo "[ SUCCESS ] Manifesto saved to $FILE."
echo ""
echo "Here is your generated manifesto:"
echo "---------------------------------"
cat $FILE
echo "---------------------------------"
```

```
admin@fedora:~/Desktop/scripts$ ./manifesto.sh
===================================
       MANIFESTO GENERATOR
===================================
Generating Open Source Manifesto...
[ SUCCESS ] Manifesto saved to foss_manifesto.txt.

Here is your generated manifesto:
---------------------------------
THE OPEN SOURCE MANIFESTO
-------------------------
1. Free as in Freedom: Users have the liberty to run, copy, distribute, study, change, and improve the software.
2. Community Driven: The power of the software comes from the collective effort of developers worldwide.
3. Transparency: Code is open to audit, ensuring better security and no hidden backdoors.
4. No Corporate Monopolies: Software should not be locked behind proprietary walls that stifle innovation.
---------------------------------
```